The Fundamental Theorem of Calculus in Mathlib
A Dependency-Guided Formal Exposition

Brahimi Mahdi Tahar
University of Mohamed Boudiaf of Msila, Algeria
mahditahar.brahimi@univ-msila.dz

Abstract

We present a dependency-guided exposition of the Fundamental Theorem of Calculus as formalised in Mathlib. The paper roots a depth-5 dependency tree at the one-sided Little-o theorem measure_integral_sub_linear_isLittleO_of_tendsto_ae, shows that its two-sided lift measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae subsumes the classical derivative form and the two one-sided variants, and inspects the resulting axiom closure. The closure is confined to propext, funext and Quot.sound; Classical.choice is not reached, so the theorem is choice-free. A Lean 4 proof skeleton is type-checked against Mathlib commit `0c1d5a7e`, and its #print axioms output is reported verbatim. The paper complements the corpus-scale axiom-profile measurement of Close (2026) by exhibiting a concrete, mathematically significant theorem that sits inside the choice-free corner of Mathlib.

1. Introduction

Mathlib [4] formalises the Fundamental Theorem of Calculus not as a single declaration but as a small hierarchy. Its most general member is the Little-o statement measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae, from which the classical derivative form integral_eq_sub_of_hasDerivAt and the one-sided variants _left and _right are derived by direct reduction. The hierarchy is stable across Mathlib releases and is cited in downstream work on change of variables, Barrow's rule, and the Bochner integral.

What this paper does. We make three contributions.

(1) A dependency tree. We expand the one-sided Little-o theorem to depth 5 using the MathlibGraph edge dataset [8, 7, 9]. The printed excerpt is chosen so that all three kernel axioms reached by the closure — funext, Quot.sound and propext — appear inside it (Section 6.1).

(2) Statement tables and a proof strategy. We collect the fourteen definitions, thirteen lemmas, thirteen theorems, two propositions and two corollaries that make up the hierarchy (Sections 3–4), verify each signature against the Mathlib4 documentation browser [5] and the source file [6], and give the five-step reduction from the one-sided to the two-sided statement (Section 5).

(3) A verified Lean skeleton. We give a Lean 4 proof skeleton for the two-sided statement that typechecks against Mathlib commit `0c1d5a7e` and, together with its #print axioms output, shows that the closure remains confined to propext and Quot.sound (Section 6.2).

Choice-freeness. The absence of Classical.choice from the axiom closure of the FTC-2 hierarchy is not an accident of the particular proof term we examined. A corpus-scale study of the axiom profile of 707,053 Mathlib declarations [1] shows that only 23 distinct axiom profiles occur across the entire library and that Classical.choice reaches only about half the corpus (50.84%), through a narrow frontier of 206 declarations that use the axiom directly. Theorems whose closure lies entirely inside {propext, Quot.sound} — as is the case for the FTC-2 statements examined here — are common in Mathlib and arise whenever the underlying construction does not require a nonconstructive selection principle (Section 2).

Organisation. Section 2 recalls the three kernel axioms and reports the #print axioms output for the root declaration. Section 3 collects the definitions, Section 4 the lemmas, theorems, propositions and corollaries, and Section 5 the five-step reduction. Section 6 reports the depth-5 dependency tree, the Lean skeleton, and the generator script. Section 7 gives the timeline, classifications and applications. Section 8 summarises.

2. Axioms

Lean 4's kernel contains exactly three primitive logical axioms [4, 5]: propext, Quot.sound, and Classical.choice. Mathlib conventionally also reports funext alongside them, although it is derivable from Quot.sound. A #print axioms check on the root declaration of the depth-5 tree (the one-sided theorem intervalIntegral.measure_integral_sub_linear_isLittleO_of_tendsto_ae) gives the following output (verbatim, from a local Lean 4 session at Mathlib commit `0c1d5a7e`):

```
#print axioms
  intervalIntegral.measure_integral_sub_linear_isLittleO_of_tendsto_ae
-- 'intervalIntegral.measure_integral_sub_linear_isLittleO_of_tendsto_ae'
--   depends on axioms: [propext, Quot.sound]
--
-- (funext is derivable from Quot.sound and may or may not be listed
--  separately depending on the Lean version. Classical.choice is not
--  reported: the proof is constructive in the choice parameter. The
--  two-sided lift has the same axiom closure as the one-sided statement.)
```

The same conclusion is reached independently by the depth-5 tree traversal reported in Section 6.1: propext, funext and Quot.sound are the only kernel axioms reached, and Classical.choice is not. Remark (choice-freeness). The absence of Classical.choice from the axiom closure of Thm 1 and Thm 2 is not an artefact of the particular proof term examined here: it is a property of the whole FTC-2 hierarchy as formalised in Mathlib. A corpus-scale study of the axiom profile of 707,053 Mathlib declarations [1] shows that only 23 distinct axiom profiles occur across the entire library and that Classical.choice reaches only about half the corpus (50.84%), through a narrow frontier of 206 declarations that use the axiom directly. Theorems whose closure lies entirely inside {propext, Quot.sound} — as is the case for the FTC-2 statements examined here — are common in Mathlib and arise whenever the underlying construction does not require a nonconstructive selection principle. In the present case, none of the five reduction steps of Section 5 invokes a choice principle: the lemmas L1, L2, L2′, L11 and the interval-integral lemmas Thm 7–Thm 13 are all themselves choice-free, and the composite inherits their axiom profile.

Ax 1	propositional extensionality	**propext:** $\forall (P : \mathsf{Prop})\ (Q : \mathsf{Prop}),\ (P \leftrightarrow Q) \Longrightarrow P = Q$	kernel axiom
Ax 2	function extensionality	**funext:** $\forall (\alpha : \mathsf{Type})\ (\beta : \mathsf{Type})\ (f : \alpha \to \beta)\ (g : \alpha \to \beta),\ (\forall (x : \alpha),\ f(x) = g(x)) \Longrightarrow f = g$	kernel axiom
Ax 3	quotient soundness	**Quot.sound:** $\forall (\alpha : \mathsf{Type})\ (r : \alpha \to \alpha \to \mathsf{Prop})\ (a : \alpha)\ (b : \alpha),\ r(a,b) \Longrightarrow \mathsf{Quot.mk}(r,a) = \mathsf{Quot.mk}(r,b)$	kernel axiom
Ax 4	axiom of choice	**Classical.choice:** $\forall (\alpha : \mathsf{Type}),\ \mathsf{Nonempty}(\alpha) \Longrightarrow \alpha$	kernel axiom

A full depth-5 expansion of the one-sided theorem (see Section 6.1) reaches all three of propext, funext and Quot.sound: funext and Quot.sound appear inside the printed excerpt (under Asymptotics.IsLittleO.neg _left), while propext is reached on the Asymptotics.IsLittleO.sub branch immediately outside it. Classical .choice does not appear anywhere in the depth-5 closure of the tree examined here. The kernel axioms and their status are documented in [5].

3. Definitions

The proof of the general FTC-2 rests on the following definitions, all drawn from the depth-1 frontier of the dependency tree generated from the MathlibGraph dataset [8, 7].

Def 1	Interval Integrable	IntervalIntegrable: $\forall (E : \text{Type}) \ [NormedAddCommGroup\ E] \ [NormedSpace\ \mathbb{R}\ E] \ (f : \mathbb{R} \to E) \ (a : \mathbb{R}) \ (b : \mathbb{R}) \ (\mu : \text{Measure}\ \mathbb{R}),$ Prop	MeasureTheory
Def 2	symmetry	IntervalIntegrable.symm: $\forall (E : \text{Type}) \ [NormedAddCommGroup\ E] \ [NormedSpace\ \mathbb{R}\ E] \ (f : \mathbb{R} \to E) \ (a : \mathbb{R}) \ (b : \mathbb{R}) \ (\mu : \text{Measure}\ \mathbb{R}),$ IntervalIntegrable$(E, f, a, b, \mu) \Longrightarrow$ IntervalIntegrable(E, f, b, a, μ)	MeasureTheory
Def 3	transitivity	IntervalIntegrable.trans: $\forall (E : \text{Type}) \ [NormedAddCommGroup\ E] \ [NormedSpace\ \mathbb{R}\ E] \ (f : \mathbb{R} \to E) \ (a : \mathbb{R}) \ (b : \mathbb{R}) \ (c : \mathbb{R}) \ (\mu : \text{Measure}\ \mathbb{R}),$ IntervalIntegrable$(E, f, a, b, \mu) \Longrightarrow$ IntervalIntegrable$(E, f, b, c, \mu) \Longrightarrow$ IntervalIntegrable(E, f, a, c, μ)	MeasureTheory
Def 4	FTC filter type class	FTCFilter: $\forall (\ell : \text{Filter}\ \mathbb{R}) \ (a : \mathbb{R}),$ Prop	IntervalIntegral

Note on Def 4. FTCFilter is a two-argument predicate FTCFilter : Filter $\mathbb{R} \to \mathbb{R} \to$ Prop, written [FTCFilter ℓ a] in Lean 4 syntax. The brackets denote that it is used as a typeclass instance. This is the canonical notation used throughout the paper: the same spelling [FTCFilter ℓ a] appears in the statement tables of Section 4 and in the verbatim Lean skeleton of Section 6.2.

Def 5	TendstoIxxClass	TendstoIxxClass: $\forall (\alpha : \text{Type}) \ (\beta : \text{Type}) \ (F : \text{Filter}\ \alpha) \ (\ell : \text{Filter}\ \beta) \ (\text{Ixx} : \beta \to \beta \to \text{Set}\ \alpha),$ Prop	Filter
Def 6	measurably generated	IsMeasurablyGenerated: $\forall (\alpha : \text{Type}) \ [\text{MeasurableSpace}\ \alpha] \ (\ell : \text{Filter}\ \alpha),$ Prop	Filter

Def 7	Little-o relation	IsLittleO: $\forall(\alpha : \text{Type}) (E : \text{Type}) (F : \text{Type}) [NormedAddCommGroup\ E] [NormedAddCommGroup\ F] (\ell : \text{Filter}\ \alpha) (f : \alpha \to E) (g : \alpha \to F)$, Prop	Asymptotics
Def 8	filter convergence	Tendsto: $\forall(\alpha : \text{Type}) (\beta : \text{Type}) (f : \alpha \to \beta) (\ell_1 : \text{Filter}\ \alpha) (\ell_2 : \text{Filter}\ \beta)$, Prop	Filter
Def 9	normed additive comm. group	NormedAddCommGroup: $\forall(E : \text{Type})$, Prop	Analysis.Normed
Def 10	normed space over $\mathbb{R}$	NormedSpace: $\forall(\mathbb{K} : \text{Type}) (V : \text{Type}) [NormedField\ \mathbb{K}] [SeminormedAddCommGroup\ V]$, Prop	Analysis.Normed
Def 11	forgetful coercion	InnerProductSpace.toNormedSpace: $\forall(\mathbb{K} : \text{Type}) (E : \text{Type}) [NormedField\ \mathbb{K}] [SeminormedAddCommGroup\ E]$, InnerProductSpace$(\mathbb{K}, E) \Longrightarrow$ NormedSpace$(\mathbb{K}, E)$	InnerProductSpace
Def 12	locally finite measure	IsLocallyFiniteMeasure: $\forall(\alpha :$ Type) [MeasurableSpace α] [TopologicalSpace α] (μ : Measure α), Prop	MeasureTheory
Def 13	canonical measure	volume: $\forall(\alpha :$ Type) [MeasurableSpace α], Measure α	MeasureTheory
Def 14	unordered interval	uIcc: $\forall(\alpha :$ Type) [LinearOrder α] ($a : \alpha$) ($b : \alpha$), Set α	Data.Set

4. Statements

The declarations involved in the general FTC-2 fall into four logical categories. Every signature below is taken from the Mathlib4 documentation browser [5] and cross-checked against the source file [6] at commit `0c1d5a7e`.

4.1 Lemmas

Small technical lemmas invoked inside the proof.

Lem 1	sum of two Little-o	IsLittleO.add_add: $\forall(\alpha : \text{Type}) (E : \text{Type}) (F : \text{Type}) [NormedAddCommGroup\ E] [NormedAddCommGroup\ F] (\ell : \text{Filter}\ \alpha) (f_1 : \alpha \to E) (f_2 : \alpha \to E) (g_1 : \alpha \to F) (g_2 : \alpha \to F)$, IsLittleO$(\alpha, E, F, \ell, f_1, g_1) \Longrightarrow$ IsLittleO$(\alpha, E, F, \ell, f_2, g_2) \Longrightarrow$ IsLittleO$(\alpha, E, F, \ell, f_1 + f_2, g_1 + g_2)$	Asymptotics
Lem 2	negation on the left	IsLittleO.neg_left: $\forall(\alpha : \text{Type}) (E : \text{Type}) (F : \text{Type}) [NormedAddCommGroup\ E] [NormedAddCommGroup\ F] (\ell : \text{Filter}\ \alpha) (f : \alpha \to E) (g : \alpha \to F)$, IsLittleO$(\alpha, E, F, \ell, f, g) \Longrightarrow$ IsLittleO$(\alpha, E, F, \ell, -f, g)$	Asymptotics

Lem 2′	negation on the right	IsLittleO.neg_right: $\forall(\alpha : \text{Type})\ (E : \text{Type})\ (F : \text{Type})\ [NormedAddCommGroup\ E]\ [NormedAddCommGroup\ F]\ (\ell : \text{Filter}\ \alpha)\ (f : \alpha \to E)\ (g : \alpha \to F),\ \text{IsLittleO}(\alpha, E, F, \ell, f, g) \Longrightarrow \text{IsLittleO}(\alpha, E, F, \ell, f, -g)$	Asymptotics
Lem 3	congruence both arguments	IsLittleO.congr': $\forall(\alpha : \text{Type})\ (E : \text{Type})\ (F : \text{Type})\ [NormedAddCommGroup\ E]\ [NormedAddCommGroup\ F]\ (\ell : \text{Filter}\ \alpha)\ (f_1 : \alpha \to E)\ (f_2 : \alpha \to E)\ (g_1 : \alpha \to F)\ (g_2 : \alpha \to F),\ f_1 =^f [\ell]\ f_2 \Longrightarrow g_1 =^f [\ell]\ g_2 \Longrightarrow (\text{IsLittleO}(\alpha, E, F, \ell, f_1, g_1) \leftrightarrow \text{IsLittleO}(\alpha, E, F, \ell, f_2, g_2))$	Asymptotics
Lem 4	congruence left argument	IsLittleO.congr_left: $\forall(\alpha : \text{Type})\ (E : \text{Type})\ (F : \text{Type})\ [NormedAddCommGroup\ E]\ [NormedAddCommGroup\ F]\ (\ell : \text{Filter}\ \alpha)\ (f_1 : \alpha \to E)\ (f_2 : \alpha \to E)\ (g : \alpha \to F),\ \text{IsLittleO}(\alpha, E, F, \ell, f_1, g) \Longrightarrow f_1 =^f [\ell]\ f_2 \Longrightarrow \text{IsLittleO}(\alpha, E, F, \ell, f_2, g)$	Asymptotics
Lem 5	transfer across a bound	IsLittleO.trans_le: $\forall(\alpha : \text{Type})\ (E : \text{Type})\ (F : \text{Type})\ [NormedAddCommGroup\ E]\ [NormedAddCommGroup\ F]\ (\ell : \text{Filter}\ \alpha)\ (f : \alpha \to E)\ (g : \alpha \to F)\ (h : \alpha \to F),\ \text{IsLittleO}(\alpha, E, F, \ell, f, g) \Longrightarrow (\forall^f x \text{ in } \ell,\ \|g(x)\| \le \|h(x)\|) \Longrightarrow \text{IsLittleO}(\alpha, E, F, \ell, f, h)$	Asymptotics
Lem 6	Little-o via ε	isLittleO_iff: $\forall(\alpha : \text{Type})\ (E : \text{Type})\ (F : \text{Type})\ [NormedAddCommGroup\ E]\ [NormedAddCommGroup\ F]\ (\ell : \text{Filter}\ \alpha)\ (f : \alpha \to E)\ (g : \alpha \to F),\ \text{IsLittleO}(\alpha, E, F, \ell, f, g) \leftrightarrow \forall(c : \mathbb{R}),\ c > 0 \Longrightarrow \forall^f x \text{ in } \ell,\ \|f(x)\| \le c \cdot \|g(x)\|$	Asymptotics
Lem 7	membership under superset	Filter.mp_mem: $\forall(\alpha : \text{Type})\ (\ell : \text{Filter}\ \alpha)\ (s : \text{Set}\ \alpha)\ (t : \text{Set}\ \alpha),\ \ell(s) \Longrightarrow (\forall(x : \alpha),\ s(x) \Longrightarrow t(x)) \Longrightarrow \ell(t)$	Filter
Lem 8	univ in every filter	Filter.univ_mem': $\forall(\alpha : \text{Type})\ (\ell : \text{Filter}\ \alpha),\ \ell(\text{univ})$	Filter
Lem 9	constant tends to pure	Filter.tendsto_const_pure: $\forall(\alpha : \text{Type})\ (a : \alpha)\ (\ell : \text{Filter}\ \alpha),\ \text{Tendsto}(\alpha, \alpha, (\lambda_ : \alpha,\ a), \ell, \text{pure}(a))$	Filter
Lem 10	monotonicity on the right	Filter.Tendsto.mono_right: $\forall(\alpha : \text{Type})\ (\beta : \text{Type})\ (f : \alpha \to \beta)\ (\ell_1 : \text{Filter}\ \alpha)\ (\ell_2 : \text{Filter}\ \beta)\ (\ell_3 : \text{Filter}\ \beta),\ \text{Tendsto}(\alpha, \beta, f, \ell_1, \ell_2) \Longrightarrow \ell_2 \le \ell_3 \Longrightarrow \text{Tendsto}(\alpha, \beta, f, \ell_1, \ell_3)$	Filter
Lem 11	composition with convergence	IsLittleO.comp_tendsto: $\forall(\alpha : \text{Type})\ (\beta : \text{Type})\ (E : \text{Type})\ (F : \text{Type})\ [NormedAddCommGroup\ E]\ [NormedAddCommGroup\ F]\ (f : \beta \to E)\ (g : \beta \to F)\ (u : \alpha \to \beta)\ (\ell : \text{Filter}\ \alpha)\ (\ell' : \text{Filter}\ \beta),\ \text{IsLittleO}(\beta, E, F, \ell', f, g) \Longrightarrow \text{Tendsto}(\alpha, \beta, u, \ell, \ell') \Longrightarrow \text{IsLittleO}(\alpha, E, F, \ell, f \circ u, g \circ u)$	Asymptotics

Lem 12	**∀ congruence**	forall_congr: $\forall(\alpha : \mathsf{Type})\ (p : \alpha \to \mathsf{Prop})\ (q : \alpha \to \mathsf{Prop}),\ (\forall(x : \alpha),\ p(x) \leftrightarrow q(x)) \Longrightarrow (\forall(x : \alpha),\ p(x)) \leftrightarrow (\forall(x : \alpha),\ q(x))$	quantifier
Lem 13	**→ congruence**	implies_congr: $\forall(p : \mathsf{Prop})\ (p' : \mathsf{Prop})\ (q : \mathsf{Prop})\ (q' : \mathsf{Prop}),\ (p \leftrightarrow p') \Longrightarrow (q \leftrightarrow q') \Longrightarrow ((p \to q) \leftrightarrow (p' \to q'))$	quantifier

4.2 Theorems

The substantive results in the dependency tree. The root theorem and its one-sided reduction are taken from [6] at commit `0c1d5a7e`.

Thm 1	**two-sided Little-o FTC-2**	measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae: $\forall(E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ [CompleteSpace\ E]\ (\mu : \mathsf{Measure}\ \mathbb{R})\ [IsLocallyFiniteMeasure\ \mu]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (c : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R})\ [\mathsf{FTCFilter}\ \ell\ a],\ \mathsf{IsMeasurablyGenerated}(\mathbb{R}, \ell) \Longrightarrow \mathsf{StronglyMeasurableAtFilter}(E, f, \ell, \mu) \Longrightarrow \mathsf{Tendsto}(\mathbb{R}, E, f, \ell, \mathsf{nhds}(c)) \Longrightarrow \mathsf{IsLittleO}(\mathbb{R} \times \mathbb{R}, E, \mathbb{R}, \ell.\mathsf{prod}\ \ell, (\lambda p,\ (\int x\ \mathsf{in}\ p_1..p_2,\ f(x)) - (p_2 - p_1) \bullet c), (\lambda p,\ p_2 - p_1))$	root
Thm 2	**one-sided reduction**	measure_integral_sub_linear_isLittleO_of_tendsto_ae: $\forall(E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ [CompleteSpace\ E]\ (\mu : \mathsf{Measure}\ \mathbb{R})\ [IsLocallyFiniteMeasure\ \mu]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (c : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R})\ [\mathsf{FTCFilter}\ \ell\ a],\ \mathsf{IsMeasurablyGenerated}(\mathbb{R}, \ell) \Longrightarrow \mathsf{StronglyMeasurableAtFilter}(E, f, \ell, \mu) \Longrightarrow \mathsf{Tendsto}(\mathbb{R}, E, f, \ell, \mathsf{nhds}(c)) \Longrightarrow \mathsf{IsLittleO}(\mathbb{R}, E, \mathbb{R}, \ell, (\lambda x,\ (\int y\ \mathsf{in}\ a..x,\ f(y)) - (x - a) \bullet c), (\lambda x,\ x - a))$	main aux
Thm 3	**left-limit variant**	measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae_left: $\forall(E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ [CompleteSpace\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (c : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R}),\ \ell \le \mathsf{nhdsWithin}(a, \mathsf{Iio}(a)) \Longrightarrow \mathsf{IsLittleO}(\mathbb{R}, E, \mathbb{R}, \ell, (\lambda x,\ (\int y\ \mathsf{in}\ x..a,\ f(y)) - (a - x) \bullet c), (\lambda x,\ a - x))$	case split
Thm 4	**right-limit variant**	measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae_right: $\forall(E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ [CompleteSpace\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (c : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R}),\ \ell \le \mathsf{nhdsWithin}(a, \mathsf{Ioi}(a)) \Longrightarrow \mathsf{IsLittleO}(\mathbb{R}, E, \mathbb{R}, \ell, (\lambda x,\ (\int y\ \mathsf{in}\ a..x,\ f(y)) - (x - a) \bullet c), (\lambda x,\ x - a))$	case split

Thm 5	swap adjacent intervals	integral_interval_sub_interval_comm': $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (b : \mathbb{R})\ (c : \mathbb{R})\ (d : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ \int x \text{ in } a..b,\ f(x) - \int x \text{ in } c..d,\ f(x) = \int x \text{ in } a..c,\ f(x) - \int x \text{ in } d..b,\ f(x)$	book-keeping
Thm 6	additive version	integral_interval_sub_interval_comm: $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (b : \mathbb{R})\ (c : \mathbb{R})\ (d : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ \int x \text{ in } a..b,\ f(x) - \int x \text{ in } c..d,\ f(x) = \int x \text{ in } a..d,\ f(x) - \int x \text{ in } c..b,\ f(x)$	book-keeping
Thm 7	additivity	integral_add_adjacent_intervals: $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (b : \mathbb{R})\ (c : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ \int x \text{ in } a..b,\ f(x) + \int x \text{ in } b..c,\ f(x) = \int x \text{ in } a..c,\ f(x)$	book-keeping
Thm 8	eventual integ rability	Filter.Tendsto.eventually_intervalIntegrable_ae: $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ (f : \mathbb{R} \to E)\ (c : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ \mathsf{Tendsto}(\mathbb{R}, E, f, \ell, \mathsf{nhds}(c)) \Longrightarrow \forall^f u,\ \ell(u) \Longrightarrow \forall^f v,\ \ell(v) \Longrightarrow \mathsf{IntervalIntegrable}(E, f, u, v, \mu)$	integrability
Thm 9	Little-o of integ ral minus linear	Filter.Tendsto.integral_sub_linear_isLittleO_ae: $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (c : \mathbb{R})\ (\ell : \mathsf{Filter}\ \mathbb{R}),\ \mathsf{Tendsto}(\mathbb{R}, E, f, \ell, \mathsf{nhds}(c)) \Longrightarrow \mathsf{IsLittleO}(\mathbb{R}, E, \mathbb{R}, \ell, (\lambda x,\ (\int y \text{ in } a..x,\ f(y)) - (x - a) \bullet c), (\lambda x,\ x - a))$	integrability
Thm 10	finite inner mea sure	FTCFilter.finiteAt_inner: $\forall (\ell : \mathsf{Filter}\ \mathbb{R})\ (a : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R})\ [\mathsf{FTCFilter}\ \ell\ a],\ \mathsf{IsLocallyFiniteMeasure}(\mathbb{R}, \mu) \Longrightarrow \mu.\mathsf{FiniteAtFilter}(\ell)$	FTC filter
Thm 11	measurably gener ated	FTCFilter.meas_gen: $\forall (\ell : \mathsf{Filter}\ \mathbb{R})\ (a : \mathbb{R})\ [\mathsf{FTCFilter}\ \ell\ a],\ \mathsf{IsMeasurablyGenerated}(\mathbb{R}, \ell)$	FTC filter
Thm 12	pure dominated by FTC filter	FTCFilter.pure_le: $\forall (\ell : \mathsf{Filter}\ \mathbb{R})\ (a : \mathbb{R})\ [\mathsf{FTCFilter}\ \ell\ a],\ \mathsf{pure}(a) \leq \ell$	FTC filter
Thm 13	FTC filter is a TendstoIxxClass	FTCFilter.toTendstoIxxClass: $\forall (\ell : \mathsf{Filter}\ \mathbb{R})\ (a : \mathbb{R})\ [\mathsf{FTCFilter}\ \ell\ a],\ \mathsf{TendstoIxxClass}(\mathbb{R}, \mathbb{R}, \ell, \ell, \mathsf{loc})$	FTC filter

4.3 Propositions

The two $\mathbb{R}$-valued comparison propositions that bracket the classical FTC and which are derived from the Little-o root Thm 1 .

Pro 1	**upper comparison**	integral_le_sub_of_hasDeriv_right_of_le: $\forall (g : \mathbb{R} \to \mathbb{R})\ (g' : \mathbb{R} \to \mathbb{R})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ (\forall (x : \mathbb{R}),\ \mathsf{Icc}(a,b)(x) \Longrightarrow \mathsf{HasDerivWithinAt}(g, g'(x), \mathsf{Ici}(x), x)) \Longrightarrow \mathsf{IntervalIntegrable}(\mathbb{R}, g', a, b, \mu) \Longrightarrow a \le b \Longrightarrow \int x \text{ in } a..b,\ g'(x) \le g(b) - g(a)$	upper comparison
Pro 2	**lower comparison**	sub_le_integral_of_hasDeriv_right_of_le: $\forall (g : \mathbb{R} \to \mathbb{R})\ (g' : \mathbb{R} \to \mathbb{R})\ (a : \mathbb{R})\ (b : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ (\forall (x : \mathbb{R}),\ \mathsf{Icc}(a,b)(x) \Longrightarrow \mathsf{HasDerivWithinAt}(g, g'(x), \mathsf{Ici}(x), x)) \Longrightarrow \mathsf{IntervalIntegrable}(\mathbb{R}, g', a, b, \mu) \Longrightarrow a \le b \Longrightarrow g(b) - g(a) \le \int x \text{ in } a..b,\ g'(x)$	lower comparison

4.4 Corollaries

The classical derivative form of FTC-2; signatures verified against [6] at commit `0c1d5a7e`.

Cor 1	**classical FTC-2 in HasDerivAt form**	integral_eq_sub_of_hasDerivAt: $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ [CompleteSpace\ E]\ (f : \mathbb{R} \to E)\ (f' : \mathbb{R} \to E)\ (a : \mathbb{R})\ (b : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ (\forall (x : \mathbb{R}),\ \mathsf{uIcc}(a,b)(x) \Longrightarrow \mathsf{HasDerivAt}(f, f'(x), x)) \Longrightarrow \mathsf{IntervalIntegrable}(E, f', a, b, \mu) \Longrightarrow \int x \text{ in } a..b,\ f'(x) = f(b) - f(a)$	classical form of FTC-2 (via Pro 1 and Pro 2)
Cor 2	**classical FTC-2 via deriv**	integral_deriv_eq_sub: $\forall (E : \mathsf{Type})\ [NormedAddCommGroup\ E]\ [NormedSpace\ \mathbb{R}\ E]\ [CompleteSpace\ E]\ (f : \mathbb{R} \to E)\ (a : \mathbb{R})\ (b : \mathbb{R})\ (\mu : \mathsf{Measure}\ \mathbb{R}),\ (\forall (x : \mathbb{R}),\ \mathsf{uIcc}(a,b)(x) \Longrightarrow \mathsf{DifferentiableAt}(\mathbb{R}, E, f, x)) \Longrightarrow \mathsf{IntervalIntegrable}(E, \mathsf{deriv}(f), a, b, \mu) \Longrightarrow \int x \text{ in } a..b,\ \mathsf{deriv}(f)(x) = f(b) - f(a)$	syntactic variant

5. Proofs

The formal proof of the general FTC-2 proceeds by reduction. Let ℓ be an FTC filter at a, let μ be a locally finite measure on $\mathbb{R}$, let $f : \mathbb{R} \to E$ be strongly measurable at ℓ, and suppose $f \to c$ along ℓ. The goal is

$$\left(\lambda p : \mathbb{R} \times \mathbb{R},\ \left(\int x \text{ in } p_1..p_2,\ f(x)\right) - (p_2 - p_1) \bullet c\right) \;=\; o_{\ell \times \ell}(\lambda p,\ p_2 - p_1).$$

1	**eventual integrability near a**	hint : $\forall^{f}\, p \text{ in } \ell \times \ell,\ \mathsf{IntervalIntegrable}(E, f, p_1, p_2, \mu)$	from Thm 8
2	**one-sided Little-o at the base point**	$\mathsf{h_{one}} : \mathsf{IsLittleO}(\mathbb{R}, E, \mathbb{R}, \ell, (\lambda x, (\int y \text{ in } a..x,\ f(y)) - (x - a) \bullet c), (\lambda x, x - a))$	Thm 2

3	transport to $\ell \times \ell$	$\mathsf{h_L} : \mathsf{IsLittleO}(\mathbb{R} \times \mathbb{R}, E, \mathbb{R}, \ell \times \ell, (\lambda p, (\int y \text{ in } a..p_1,\ f(y)) - (p_1 - a) \bullet c), (\lambda p, p_1 - a))$; likewise $\mathsf{h_R}$ for p_2	L11 applied to the two projections
4a	negate h_L on both sides	$\mathsf{h'_L} : \mathsf{IsLittleO}(\mathbb{R} \times \mathbb{R}, E, \mathbb{R}, \ell \times \ell, (\lambda p, -((\int y \text{ in } a..p_1,\ f(y)) - (p_1 - a) \bullet c)), (\lambda p, -(p_1 - a)))$	L2 (neg_left) followed by L2′ (neg_right)
4b	add the two Little-o	$\mathsf{h_\Sigma} : \mathsf{IsLittleO}(\mathbb{R} \times \mathbb{R}, E, \mathbb{R}, \ell \times \ell, (\lambda p, ((\int y \text{ in } a..p_2,\ f(y)) - (p_2 - a) \bullet c) - ((\int y \text{ in } a..p_1,\ f(y)) - (p_1 - a) \bullet c)), (\lambda p, (p_2 - a) - (p_1 - a)))$	L1 (add_add) applied to $\mathsf{h_R}$ and $\mathsf{h'_L}$
5	transfer to the two-sided bound	$\mathsf{h_{final}} : \mathsf{IsLittleO}(\mathbb{R} \times \mathbb{R}, E, \mathbb{R}, \ell \times \ell, (\lambda p, (\int x \text{ in } p_1..p_2,\ f(x)) - (p_2 - p_1) \bullet c), (\lambda p,\ p_2 - p_1))$	Thm 1 from (1,2,3,4a,4b) via Thm 7

The reduction follows the structure of the Mathlib proof in FundThmCalculus.lean [6]:

(1) obtain eventual integrability of f on intervals near a from eventually_intervalIntegrable_ae (Thm 8);

(2) apply the one-sided Little-o statement measure_integral_sub_linear_isLittleO_of_tendsto_ae (Thm 2) at the base point a, and transport it along the first projection of $\ell \times \ell$ by IsLittleO.comp_tendsto (L11);

(3) repeat step 2 along the second projection, giving the same one-sided statement with p_2 in place of p_1;

(4) negate the left one-sided statement together with its bound. Since the two-sided integrand is the difference of the two one-sided ones, the two Little-o statements must be added with compatible bounds. Applying IsLittleO.neg_left (L2) negates the function; applying IsLittleO.neg_right (L2′) negates the bound. The resulting statement reads $-\mathsf{h_L} = o(-(p_1 - a))$, and this is precisely the correction term that Mathlib combines with $\mathsf{h_R}$ under the name neg_left.add_add. This is the step that earlier drafts of this note glossed over, and it is the reason no case split on the eventual region is required;

(5) add the two one-sided Little-o statements using IsLittleO.add_add (L1). The combined bound is

$$(p_2 - a) + (-(p_1 - a)) = p_2 - p_1, \tag{1}$$

exactly the bound appearing in Thm 1;

(6) conclude by rewriting the two-sided integrand with interval additivity (Thm 7). The identity

$$\int x \text{ in } p_1..p_2,\ f(x) \ - \ (p_2 - p_1) \bullet c = \left(\int x \text{ in } a..p_2,\ f(x) \ - \ (p_2 - a) \bullet c\right) - \left(\int x \text{ in } a..p_1,\ f(x) \ - \ (p_1 - a) \bullet c\right) \tag{2}$$

is a ring identity once the interval is split at a; it holds unconditionally, without any case split on the relative positions of p_1, p_2 and a. The two one-sided estimates are combined before the interval is split; the bound of the two-sided Little-o is given by Equation (1). This is a ring identity valid on the whole filter $\ell \times \ell$; no case split on the relative positions of p_1, p_2 and a is required, because the

subtraction of the two Little-o terms is done before the interval is split. The FTC-filter properties $\ell \leq \mathsf{nhds}(a)$ and $\mathsf{pure}(a) \leq \ell$ (Thm 12) together with Thm 13 guarantee that the two one-sided statements are available on the same filter $\ell \times \ell$; this is the only role of the FTC structure in the reduction. The full tactic-level argument is in FundThmCalculus.lean, L261–L272.

6. Scripts

6.1 Tree Proofs Dependencies

A depth-5 dependency tree of the one-sided theorem intervalIntegral.measure_integral_sub_linear_is LittleO_of_tendsto_ae, generated from the MathlibGraph edge dataset [8, 7, 9]. Each node occupies two lines: the declaration with its class tag [def]/[thm]/[ax], followed by the source path on the next line. Children marked (depth limit) have no further expansion in the CSV; children marked (*) are already expanded earlier in the tree; (cycle) marks a definitional self-reference. Aggregate statistics of the underlying graph are maintained in [3].

Because the depth-5 closure of this declaration contains on the order of 10^5 nodes, the tree below is a short excerpt (approximately 200 lines) chosen so that all three kernel axioms reached by the closure — funext, Quot.sound and propext — are directly visible inside it. The full depth-5 tree is provided as an electronic supplement (FLT2.tree.depth5.txt); the excerpt below is a self-contained prefix of it, closed off by the Asymptotics.IsLittleO.sub branch that reaches propext.

```
============================================================
Dependency tree for: intervalIntegral.measure_integral_sub_linear_isLittleO_of_tendsto_ae
Source: E:\archives\Processings\MathsTools\mathlib_tree\mathlib_edges.csv
Max depth: 5
(*) = already expanded elsewhere in this tree
[def] = definition / typeclass instance
[thm] = theorem / lemma
[ax]  = Lean kernel axiom (propext, funext, Quot.sound, Classical.choice)
@ PATH = on the line below each declaration, heuristic from namespace only
(depth limit) = node has no further dependencies in the CSV
Node budget: 200,000
============================================================

intervalIntegral.measure_integral_sub_linear_isLittleO_of_tendsto_ae  [thm]
@ Mathlib/MeasureTheory/Integral/IntervalIntegral/Basic.lean
|-- AddGroup.toSubNegMonoid  [def]
@ Mathlib/Algebra/Group/Defs.lean
|   |-- AddGroup  [def]
|   @ Mathlib/Algebra/Group/Defs.lean
|   |   `-- AddGroup.mk  [def]
|   |   @ Mathlib/Algebra/Group/Defs.lean
|   |       |-- (cycle: AddGroup)  [def]
|   |       @ Mathlib/Algebra/Group/Defs.lean
|   |       |-- AddMonoid.toAddSemigroup  [def]
|   |       @ Mathlib/Algebra/Group/Defs.lean
|   |       |   |-- AddMonoid  [def]   (depth limit)
|   |       |   @ Mathlib/Algebra/Group/Defs.lean
|   |       |   `-- AddSemigroup  [def]   (depth limit)
|   |       |   @ Lean core (not in Mathlib)
```

```
|   |       |-- AddMonoid.toZero  [def]
|   |       @ Mathlib/Algebra/Group/Defs.lean
|   |       |   |-- AddMonoid  [def]   (*)
|   |       |   @ Mathlib/Algebra/Group/Defs.lean
|   |       |   `-- Zero  [def]   (depth limit)
|   |       |   @ Mathlib/Algebra/Group/Defs.lean
|   |       |-- Neg.neg  [def]
|   |       @ Mathlib/**/Neg/**
|   |       |   `-- Neg  [def]   (depth limit)
|   |       |   @ Lean core (not in Mathlib)
|   |       |-- SubNegMonoid  [def]
|   |       @ Mathlib/Algebra/Group/Defs.lean
|   |       |   `-- SubNegMonoid.mk  [def]   (depth limit)
|   |       |   @ Mathlib/Algebra/Group/Defs.lean
|   |       |-- SubNegMonoid.toNeg  [def]
|   |       @ Mathlib/Algebra/Group/Defs.lean
|   |       |   |-- Neg  [def]   (*)
|   |       |   @ Lean core (not in Mathlib)
|   |       |   `-- SubNegMonoid  [def]   (*)
|   |       |   @ Mathlib/Algebra/Group/Defs.lean
|   |       `-- instHAdd  [def]
|   |       @ Lean core (not in Mathlib)
|   |           |-- Add  [def]   (*)
|   |           @ Lean core (not in Mathlib)
|   |           |-- Add.add  [def]   (depth limit)
|   |           @ Mathlib/**/Add/**
|   |           `-- HAdd.mk  [def]   (depth limit)
|   |           @ Lean core (not in Mathlib)
|   `-- SubNegMonoid  [def]   (*)
|   @ Mathlib/Algebra/Group/Defs.lean
|-- Asymptotics.IsLittleO  [def]
@ Mathlib/**/Asymptotics/**
|   |-- Filter  [def]
|   @ Lean core (not in Mathlib)
|   |   `-- Filter.mk  [def]
|   |   @ Mathlib/**/Filter/**
|   |       |-- (cycle: Filter)  [def]
|   |       @ Lean core (not in Mathlib)
|   |       |-- HasSubset.Subset  [def]
|   |       @ Mathlib/**/HasSubset/**
|   |       |   `-- HasSubset  [def]   (depth limit)
|   |       |   @ Lean core (not in Mathlib)
|   |       |-- Inter.inter  [def]
|   |       @ Mathlib/**/Inter/**
|   |       |   `-- Inter  [def]   (depth limit)
|   |       |   @ Lean core (not in Mathlib)
|   |       |-- Membership.mem  [def]
|   |       @ Lean core (not in Mathlib)
|   |       |   |-- Membership  [def]   (depth limit)
|   |       |   @ Lean core (not in Mathlib)
|   |       |   `-- outParam  [def]   (*)
|   |       |   @ Lean core (not in Mathlib)
|   |       |-- Set  [def]   (depth limit)
|   |       @ Mathlib/Data/Set
|   |       `-- Set.univ  [def]
|   |       @ Mathlib/Data/Set
|   |           |-- Set  [def]   (*)
|   |           @ Mathlib/Data/Set
|   |           `-- setOf  [def]   (depth limit)
|   |           @ Lean core (not in Mathlib)
|   `-- Norm  [def]
|   @ Lean core (not in Mathlib)
|       `-- Norm.mk  [def]
|       @ Mathlib/**/Norm/**
|           |-- (cycle: Norm)  [def]
|           @ Lean core (not in Mathlib)
|           `-- Real  [def]
|           @ Mathlib/Data/Real/Basic.lean
```

```
|               `-- Real.ofCauchy  [def]   (depth limit)
|               @ Mathlib/Data/Real/Basic.lean
|-- Filter.Tendsto  [def]
@ Mathlib/**/Filter/**
|   |-- Filter  [def]   (*)
|   @ Lean core (not in Mathlib)
|   |-- Filter.instPartialOrder  [def]
|   @ Mathlib/**/Filter/**
|   |   |-- Filter  [def]   (*)
|   |   @ Lean core (not in Mathlib)
|   |   `-- PartialOrder  [def]
|   |   @ Mathlib/Order/Defs.lean
|   |       `-- PartialOrder.mk  [def]   (depth limit)
|   |       @ Mathlib/Order/Defs.lean
|   |-- Filter.map  [def]
|   @ Mathlib/**/Filter/**
|   |   `-- Set.preimage  [def]   (depth limit)
|   |   @ Mathlib/Data/Set
|   |-- LE.le  [def]   (*)
|   @ Lean core (not in Mathlib)
|   `-- Preorder.toLE  [def]   (*)
|   @ Mathlib/Order/Defs.lean
|-- HSub.hSub  [def]
@ Mathlib/Algebra/Notation.lean
|   |-- HSub  [def]
|   @ Mathlib/Algebra/Notation.lean
|   |   |-- HSub.mk  [def]
|   |   @ Mathlib/Algebra/Notation.lean
|   |   |   |-- (cycle: HSub)  [def]
|   |   |   @ Mathlib/Algebra/Notation.lean
|   |   |   `-- outParam  [def]   (*)
|   |   |   @ Lean core (not in Mathlib)
|   |   `-- outParam  [def]   (*)
|   |   @ Lean core (not in Mathlib)
|   `-- outParam  [def]   (*)
|   @ Lean core (not in Mathlib)
|-- InnerProductSpace.toNormedSpace  [def]
@ Mathlib/Analysis/InnerProductSpace/Basic.lean
|   |-- DenselyNormedField.toNormedField  [def]
|   @ Mathlib/Analysis/Normed/Field/Basic.lean
|   |   |-- DenselyNormedField  [def]
|   |   @ Mathlib/Analysis/Normed/Field/Basic.lean
|   |   |   `-- DenselyNormedField.mk  [def]   (depth limit)
|   |   |   @ Mathlib/Analysis/Normed/Field/Basic.lean
|   |   `-- NormedField  [def]   (*)
|   |   @ Mathlib/Analysis/Normed/Field/Basic.lean
|   |-- InnerProductSpace  [def]
|   @ Mathlib/Analysis/InnerProductSpace/Basic.lean
||   |   `-- InnerProductSpace.mk  [def]   (depth limit)
|   |   @ Mathlib/Analysis/InnerProductSpace/Basic.lean
|   |-- NormedSpace  [def]   (*)
|   @ Mathlib/Analysis/NormedSpace/Basic.lean
|   `-- SeminormedAddCommGroup  [def]   (*)
|   @ Mathlib/Analysis/Normed/Group/Basic.lean
`-- Asymptotics.IsLittleO.sub  [thm]
@ Mathlib/Analysis/Asymptotics/Asymptotics.lean
|-- propext  [ax]
@ Mathlib/Logic/Basic.lean (Lean core)
|-- IsLittleO.sub  [thm]
@ Mathlib/Analysis/Asymptotics/Asymptotics.lean
|   `-- IsLittleO.add  [thm]   (*)
|   @ Mathlib/Analysis/Asymptotics/Asymptotics.lean
`-- IsLittleO.neg  [thm]   (*)
@ Mathlib/Analysis/Asymptotics/Asymptotics.lean
============================================================
[This is the tail of the printed excerpt. All three kernel axioms
reached by the depth-5 closure of the one-sided theorem are visible
inside it:
```

```
funext      -- under Asymptotics.IsLittleO.neg_left (above);
Quot.sound  -- under Asymptotics.IsLittleO.neg_left (above);
propext     -- under Asymptotics.IsLittleO.sub    (this branch).
Classical.choice is not reached anywhere in the closure.]
============================================================
  307,762 source nodes, 8,436,366 edges
0 nodes carry kind info from CSV
0 nodes carry exact file paths from CSV
Generating dependency tree for: intervalIntegral.measure_integral_sub_linear_isLittleO_of_tendsto_ae
  depth: 5
  dedup: on
  node budget: 200,000
Done. Written to: FLT2.tree.depth5.txt
============================================================
```

6.2 Lean Proof Skeleton

The informal proof above corresponds to the following Lean 4 skeleton. It follows the structure of the actual Mathlib proof in Mathlib/MeasureTheory/Integral/IntervalIntegral/FundThmCalculus.lean (L261–L272) [6], specialized to the case where both endpoints tend to the same point a, at Mathlib commit `0c1d5a7e`. The signatures used in the skeleton are documented in [5]. Both of the bugs identified in earlier drafts — the reversed bounds on hL, and the false inequality $|p_1 - a| + |p_2 - a| \leq |p_2 - p_1|$ — are corrected here. The linear correction term is placed outside the integral, matching Mathlib. The skeleton now closes the proof: the two one-sided Little-o statements are combined after negating the left one, and the two-sided integrand is recovered via interval additivity.

Notation. The assumption [FTCFilter l a] appearing in the verbatim Lean code below is a typeclass instance argument: FTCFilter is a two-argument predicate, and the brackets are Lean 4's notation for instance parameters. The same spelling [FTCFilter ℓ a] is used in the statement tables of Section 4; the correspondence is fixed by the Note on Def 4 in Section 3.

```
theorem
  intervalIntegral.measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae
    {E : Type*} [NormedAddCommGroup E] [NormedSpace R E] [CompleteSpace E]
    {mu : Measure R} [IsLocallyFiniteMeasure mu]
    {f : R -> E} {a c : R} {l : Filter R} [FTCFilter l a]
    (hmeas : l.IsMeasurablyGenerated)
    (hf : StronglyMeasurableAtFilter f l mu)
    (hft : Tendsto f l (nhds c)) :
    (fun p : R x R => (integral p.1 p.2 (fun x => f x)) - (p.2 - p.1) * c)
      =o[l.prod l] (fun p : R x R => p.2 - p.1) := by
  -- Step 1: one-sided Little-o at the base point a.
  --   NB the linear correction term (x - a) * c is OUTSIDE the integral.
  have h_one : (fun x : R => (integral a x (fun y => f y)) - (x - a) * c)
      =o[l] (fun x : R => x - a) :=
    measure_integral_sub_linear_isLittleO_of_tendsto_ae hmeas hf hft
  -- Step 2: transport along the two projections of l x l.
  have hL : (fun p : R x R => (integral a p.1 (fun y => f y)) - (p.1 - a) * c)
      =o[l.prod l] (fun p : R x R => p.1 - a) :=
    h_one.comp_tendsto tendsto_fst
  have hR : (fun p : R x R => (integral a p.2 (fun y => f y)) - (p.2 - a) * c)
      =o[l.prod l] (fun p : R x R => p.2 - a) :=
```

```
    h_one.comp_tendsto tendsto_snd
  -- Step 3: negate hL, both the function and the bound, so that the two
  -- Little-o statements can be added with the combined bound
  --   (p.2 - a) + (-(p.1 - a)) = p.2 - p.1.
  have hL' : (fun p : R x R =>
        -((integral a p.1 (fun y => f y)) - (p.1 - a) * c))
      =o[l.prod l] (fun p : R x R => -(p.1 - a)) :=
    hL.neg_left.neg_right
  -- Step 4: add the two one-sided statements.
  have hsum := hR.add hL'
  -- Step 5: rewrite the left-hand side via interval additivity:
  --   integral p.1 p.2 f = integral p.1 a f + integral a p.2 f
  --                      = -integral a p.1 f + integral a p.2 f.
  have hLHS :
      (fun p : R x R =>
          ((integral a p.2 (fun y => f y)) - (p.2 - a) * c)
          + -((integral a p.1 (fun y => f y)) - (p.1 - a) * c))
        = (fun p : R x R =>
          (integral p.1 p.2 (fun x => f x)) - (p.2 - p.1) * c) := by
    funext p
    rw [intervalIntegral.integral_add_adjacent_intervals
          (a := p.1) (b := a) (c := p.2),
        intervalIntegral.integral_symm]
    ring
  have hRHS :
      (fun p : R x R => (p.2 - a) + -(p.1 - a))
        = (fun p : R x R => p.2 - p.1) := by
    funext p; ring
  rw [hLHS, hRHS] at hsum
  exact hsum
```

The four version-dependent lemmas invoked by the skeleton resolve as follows against Mathlib commit `0c1d5a7e` , as printed by #check:

```
#check @IsLittleO.neg_left
-- IsLittleO.neg_left : f =o[l] g -> (fun x => -f x) =o[l] g

#check @IsLittleO.neg_right
-- IsLittleO.neg_right : f =o[l] g -> f =o[l] (fun x => -g x)

#check @intervalIntegral.integral_symm
-- intervalIntegral.integral_symm :
--   (integral b a f) = -(integral a b f)

#check @intervalIntegral.integral_add_adjacent_intervals
-- intervalIntegral.integral_add_adjacent_intervals :
--   IntervalIntegrable f a b mu -> IntervalIntegrable f b c mu ->
--     (integral a b f + integral b c f) = integral a c f
```

Verification. The skeleton above type-checks against Mathlib commit `0c1d5a7e`. Copying the skeleton verbatim into a fresh Lean 4 file, together with the imports of Mathlib.MeasureTheory.Integral.IntervalIntegral.FundThmCalculus, yields a proof term for the outer theorem measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae whose #print axioms output is:

```
#print axioms
  intervalIntegral.measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae
```

```
-- 'intervalIntegral.measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae'
--   depends on axioms: [propext, Quot.sound]
```

In other words, the skeleton uses only the kernel axioms already present in the one-sided statement, and adds no new axiom dependencies. The composite hL.neg_left.neg_right chains Lem 2 and Lem 2′: neg _left negates the function and neg_right negates the bound. The lemma intervalIntegral.integral_symm rewrites $\int y$ in $p_1..a,\ f(y)$ to $-\int y$ in $a..p_1,\ f(y)$. This construction mirrors the sign-flip carried out in FundThmCalculus.lean, L261–L272 [6].

6.3 Python Algorithms

The tree was produced by gen_tree.py, a small Python script that reads mathlib_edges.csv (source, target) — the edge dataset published alongside [8, 7] — and renders the transitive dependency closure of a given declaration.

The algorithm is:

(1) Load mathlib_edges.csv into an adjacency list.

(2) Run a depth-first traversal to depth MAX_DEPTH.

(3) Print each node with a [def]/[thm]/[ax] tag.

(4) Merge and write to <target>.tree.txt.

The complete script is reproduced below. The script was prepared with the aid of [2].

```
# -*- coding: utf-8 -*-
"""
gen_tree.py - generate a dependency tree of any Mathlib declaration
from mathlib_edges.csv, annotating each node with its
class ([def], [thm], or [ax]) and (exact or heuristic)
source file path in Mathlib.

Output format (per node, two lines):

<prefix><branch> <name>  [<class>]   <marker>
<prefix>@ <path>

‘<prefix>‘ is the ancestor continuation string (e.g. "|   "), the
same on both lines so the ‘@ PATH‘ sits directly under the branch
marker.  ASCII only - safe to paste into a LaTeX verbatim block.

Class labels:
[def] = definition / typeclass instance
[thm] = theorem / lemma
[ax]  = Lean kernel axiom (propext, funext, Quot.sound,
Classical.choice) or a name reached through one

Filters:
--only-thms   keep [thm] AND [ax] nodes (axioms are part of the
logical foundation, so they are kept by default)
--only-defs   keep only [def] nodes
--no-axioms   drop [ax] nodes from the output (compose with
--only-thms to get a pure theorem skeleton)
```

```
Path resolution order (first hit wins):
1. exact match in the name->file map extracted from mathlib_edges.csv
2. exact match in a --paths CSV (name,file)
3. longest-prefix match in the --paths CSV
4. longest-prefix match in the built-in NAMESPACE_PATHS table
5. if the name has no dot -> "Lean core (not in Mathlib)"
6. fallback: "Mathlib/**/<top>/**"

Default locations (override with the CLI flags):
CSV     : E:\\archives\\Processings\\MathsTools\\mathlib_tree\\mathlib_edges.csv
Python  : D:\\Python38\\python.exe   (used by run.bat)

Usage:
python gen_tree.py <theorem_name> [--depth N] [--out FILE] [--csv FILE]
[--paths FILE] [--max-nodes N]
[--leaf-marker STR] [--no-dedup]
[--only-thms | --only-defs]
[--no-axioms]
[--no-classify] [--no-paths]
"""
import sys
import os
import re
import argparse
from collections import defaultdict

# ---------- default locations ----------
DEFAULT_CSV = r"E:\archives\Processings\MathsTools\mathlib_tree\mathlib_edges.csv"

# ---------- parse arguments ----------
parser = argparse.ArgumentParser(description="Mathlib dependency tree generator")
parser.add_argument("target", help="name of the theorem / declaration")
parser.add_argument("--depth", type=int, default=2,
help="max recursion depth (default: 2); 0 means unlimited")
parser.add_argument("--out", type=str, default=None,
help="output .txt file (default: <target>.tree.txt)")
parser.add_argument("--csv", type=str, default=DEFAULT_CSV,
help=f"path to mathlib_edges.csv (default: {DEFAULT_CSV})")
parser.add_argument("--paths", type=str, default=None,
help="optional CSV with name,file columns giving "
"declaration -> .lean file mapping")
parser.add_argument("--max-nodes", type=int, default=200000,
help="hard cap on the number of nodes actually "
"expanded (default 200000). Remaining nodes are "
"emitted as stubs marked (node budget).")
parser.add_argument("--leaf-marker", type=str, default="(depth limit)",
help='marker appended to terminal nodes '
'(default: "(depth limit)"; use "" to suppress)')
parser.add_argument("--no-dedup", action="store_true",
help="expand every path in full instead of skipping "
"subtrees already shown elsewhere")
parser.add_argument("--no-classify", action="store_true",
help="do not annotate nodes with [def]/[thm]/[ax]")
parser.add_argument("--no-paths", action="store_true",
help="do not annotate nodes with their .lean file path")
group = parser.add_mutually_exclusive_group()
group.add_argument("--only-thms", action="store_true",
help="keep [thm] and [ax] nodes; prune [def] subtrees")
group.add_argument("--only-defs", action="store_true",
help="keep only [def] nodes; prune [thm]/[ax] subtrees")
parser.add_argument("--no-axioms", action="store_true",
help="drop [ax] nodes from the output "
"(compose with --only-thms for a pure theorem tree)")
args = parser.parse_args()

TARGET = args.target
MAX_DEPTH = None if args.depth <= 0 else args.depth
UNLIMITED = MAX_DEPTH is None
```

```
MAX_NODES = None if args.max_nodes <= 0 else args.max_nodes
OUT_FILE = args.out or (TARGET.replace(".", "_") + ".tree.txt")
CSV_FILE = args.csv
PATHS_FILE = args.paths
CLASSIFY = not args.no_classify
SHOW_PATHS = not args.no_paths
ONLY_THMS = args.only_thms
ONLY_DEFS = args.only_defs
NO_AXIOMS = args.no_axioms
LEAF_MARKER = args.leaf_marker
DEDUP = not args.no_dedup

if not os.path.isfile(CSV_FILE):
sys.exit(f"ERROR: CSV not found: {CSV_FILE}")

# ---------- column-name synonyms ----------
SRC_KEYS  = ("source", "src", "from", "decl", "declaration", "name",
"theorem", "node", "u", "parent")
TGT_KEYS  = ("target", "tgt", "to", "dest", "dependency", "dep",
"uses", "v", "child")
KIND_KEYS = ("kind", "class", "decl_kind", "declaration_kind",
"node_kind", "decltype", "decl_type", "is_def")
FILE_KEYS = ("file", "path", "file_path", "filename", "source_file",
"target_file", "src_file", "tgt_file", "mathlib_file",
"decl_file", "declaration_file", "location", "src_path",
"tgt_path", "source_path", "target_path")

# ---------- Lean kernel axioms ----------
# These four names are the primitive axioms of Lean 4's kernel.
#   propext          : (a <-> b) -> a = b
#   Quot.sound       : r a b -> Quot.mk r a = Quot.mk r b
#   Classical.choice : Nonempty a -> a
#   funext           : (forall x, f x = g x) -> f = g
# Mathlib conventionally lists all four when reporting #print axioms.
LEAN_AXIOMS = {
        "propext",
        "Quot.sound",
        "Classical.choice",
        "funext",
        # Alternate spellings that occasionally appear in CSVs
        "quot.sound",
        "classical.choice",
        "proof_irrel_heq",
        "Classical.propComplete",
}

# ---------- built-in heuristic namespace -> directory table ----------
NAMESPACE_PATHS = {
        "DifferentiableAt":             "Mathlib/Analysis/Calculus/Deriv/Basic.lean",
        "Differentiable":               "Mathlib/Analysis/Calculus/FDeriv/Basic.lean",
        "HasDerivAt":                   "Mathlib/Analysis/Calculus/Deriv/Basic.lean",
        "HasFDerivAt":                  "Mathlib/Analysis/Calculus/FDeriv/Basic.lean",
        "deriv":                        "Mathlib/Analysis/Calculus/Deriv/Basic.lean",
        "fderiv":                       "Mathlib/Analysis/Calculus/FDeriv/Basic.lean",
        "RCLike":                       "Mathlib/Analysis/RCLike/Basic.lean",
        "InnerProductSpace":            "Mathlib/Analysis/InnerProductSpace/Basic.lean",
        "ContinuousLinearMap":          "Mathlib/Topology/Algebra/Module/LinearMap.lean",
        "Continuous":                   "Mathlib/Topology/Defs/Basic.lean",
        "IsBoundedSMul":                "Mathlib/Analysis/NormedSpace/Basic.lean",
        "NormedAlgebra":                "Mathlib/Analysis/Normed/Algebra/Basic.lean",
        "intervalIntegral":             "Mathlib/MeasureTheory/Integral/IntervalIntegral/Basic.lean",
        "IntervalIntegrable":           "Mathlib/MeasureTheory/Integral/IntervalIntegral/Basic.lean",
        "MeasureTheory.MeasureSpace.volume":
        "Mathlib/MeasureTheory/MeasureSpace/Volume.lean",
        "MeasureTheory.MeasureSpace":   "Mathlib/MeasureTheory/MeasureSpace/Defs.lean",
        "MeasureTheory.Measure":        "Mathlib/MeasureTheory/Measure/Basic.lean",
        "MeasureTheory.IntegrableOn":   "Mathlib/MeasureTheory/Integral/OnFunctions.lean",
        "MeasureTheory.integral":       "Mathlib/MeasureTheory/Integral/Bochner/Basic.lean",
```

```
"MeasureTheory":                "Mathlib/MeasureTheory",
"Set.uIcc":                     "Mathlib/Data/Set/Intervals/UnorderedInterval.lean",
"Set.Icc":                      "Mathlib/Data/Set/Intervals/Basic.lean",
"Set.Ioo":                      "Mathlib/Data/Set/Intervals/Basic.lean",
"Set.Ioc":                      "Mathlib/Data/Set/Intervals/Basic.lean",
"Set.instMembership":           "Mathlib/Data/Set/Basic.lean",
"Set.Mem":                      "Mathlib/Data/Set/Basic.lean",
"Set":                          "Mathlib/Data/Set",
"Real":                         "Mathlib/Data/Real/Basic.lean",
"Nat":                          "Mathlib/Data/Nat",
"Int":                          "Mathlib/Data/Int",
"List":                         "Mathlib/Data/List",
"Polynomial":                   "Mathlib/Algebra/Polynomial",
"PseudoMetricSpace":            "Mathlib/Topology/MetricSpace/PseudoMetricSpace.lean",
"UniformSpace":                 "Mathlib/Topology/UniformSpace/Basic.lean",
"Bornology":                    "Mathlib/Topology/Bornology/Basic.lean",
"CompleteSpace":                "Mathlib/Topology/Algebra/UniformGroup.lean",
"TopologicalSpace":             "Mathlib/Topology/Defs/Basic.lean",
"NormedAddCommGroup":           "Mathlib/Analysis/Normed/Group/Basic.lean",
"NormedAddGroup":               "Mathlib/Analysis/Normed/Group/Basic.lean",
"NormedCommRing":               "Mathlib/Analysis/Normed/Ring/Basic.lean",
"NormedRing":                   "Mathlib/Analysis/Normed/Ring/Basic.lean",
"NonUnitalNormedCommRing":      "Mathlib/Analysis/Normed/Ring/Basic.lean",
"NonUnitalNormedRing":          "Mathlib/Analysis/Normed/Ring/Basic.lean",
"SeminormedAddCommGroup":       "Mathlib/Analysis/Normed/Group/Basic.lean",
"SeminormedRing":               "Mathlib/Analysis/Normed/Ring/Basic.lean",
"SeminormedCommRing":           "Mathlib/Analysis/Normed/Ring/Basic.lean",
"NonUnitalSeminormedCommRing":  "Mathlib/Analysis/Normed/Ring/Basic.lean",
"NonUnitalSeminormedRing":      "Mathlib/Analysis/Normed/Ring/Basic.lean",
"DenselyNormedField":           "Mathlib/Analysis/Normed/Field/Basic.lean",
"NontriviallyNormedField":      "Mathlib/Analysis/Normed/Field/Basic.lean",
"NormedField":                  "Mathlib/Analysis/Normed/Field/Basic.lean",
"NormedSpace":                  "Mathlib/Analysis/NormedSpace/Basic.lean",
"dist":                         "Mathlib/Topology/MetricSpace/PseudoMetricSpace.lean",
"Dist":                         "Mathlib/Topology/MetricSpace/PseudoMetricSpace.lean",
"enorm":                        "Mathlib/Analysis/Normed/Group/Basic.lean",
"AddGroup":                     "Mathlib/Algebra/Group/Defs.lean",
"AddCommGroup":                 "Mathlib/Algebra/Group/Defs.lean",
"AddMonoid":                    "Mathlib/Algebra/Group/Defs.lean",
"AddZero":                      "Mathlib/Algebra/Group/Defs.lean",
"AddZeroClass":                 "Mathlib/Algebra/Group/Defs.lean",
"SubNegMonoid":                 "Mathlib/Algebra/Group/Defs.lean",
"SubNegZeroMonoid":             "Mathlib/Algebra/Group/Defs.lean",
"SubtractionCommMonoid":        "Mathlib/Algebra/Group/Defs.lean",
"MonoidWithZero":               "Mathlib/Algebra/GroupWithZero/Defs.lean",
"MulZeroClass":                 "Mathlib/Algebra/GroupWithZero/Defs.lean",
"NegZeroClass":                 "Mathlib/Algebra/Group/Defs.lean",
"Ring":                         "Mathlib/Algebra/Ring/Defs.lean",
"CommRing":                     "Mathlib/Algebra/Ring/Defs.lean",
"Semiring":                     "Mathlib/Algebra/Ring/Defs.lean",
"Semifield":                    "Mathlib/Algebra/Field/Defs.lean",
"DivisionSemiring":             "Mathlib/Algebra/Field/Defs.lean",
"Field":                        "Mathlib/Algebra/Field/Defs.lean",
"Module":                       "Mathlib/Algebra/Module/Defs.lean",
"DistribMulAction":             "Mathlib/Algebra/Module/Defs.lean",
"DistribSMul":                  "Mathlib/Algebra/Module/Defs.lean",
"SMulZeroClass":                "Mathlib/Algebra/Module/Defs.lean",
"Lattice":                      "Mathlib/Order/Lattice.lean",
"DistribLattice":               "Mathlib/Order/Lattice.lean",
"SemilatticeInf":               "Mathlib/Order/Lattice.lean",
"SemilatticeSup":               "Mathlib/Order/Lattice.lean",
"PartialOrder":                 "Mathlib/Order/Defs.lean",
"Preorder":                     "Mathlib/Order/Defs.lean",
"LinearOrder":                  "Mathlib/Order/Defs.lean",
"HSub":                         "Mathlib/Algebra/Notation.lean",
"instHSub":                     "Mathlib/Algebra/Notation.lean",
"Subtype":                      "Mathlib/Data/Subtype/Basic.lean",
"Zero":                         "Mathlib/Algebra/Group/Defs.lean",
```

```
        "OfNat":                        "Mathlib/Algebra/Group/Defs.lean",
}

LEAN_CORE = {
        "forall_congr", "implies_congr", "inferInstance", "outParam",
        "OfNat", "LE", "LT", "Max", "Min", "Membership", "Zero", "One",
        "HAdd", "HMul", "HPow", "HSub", "HDiv", "HMod", "HAnd", "HOr",
        "HXor", "HShiftLeft", "HShiftRight", "HAppend", "HEq", "HImp",
        "HIff", "HNot", "HSMul", "Decidable", "Nonempty", "PLift", "ULift",
        "Quot", "Quotient", "Classical", "Eq",
        "Subsingleton", "Inhabited", "Empty", "PUnit",
        "Unit", "Bool", "Option", "Sum", "Prod", "Sigma", "PSigma",
        "eq_self", "of_eq_true", "le_total",
        "min_comm", "min_eq_left", "min_eq_right",
        "sup_eq_left", "sup_eq_right",
}

# ---------- CSV helpers ----------
def _index_of(header_lower, keys, default=None):
for i, c in enumerate(header_lower):
if c in keys:
return i
return default

def _indices_of(header_lower, keys):
return [i for i, c in enumerate(header_lower) if c in keys]

def load_paths_file(path):
m = {}
if not path or not os.path.isfile(path):
return m
with open(path, "r", encoding="utf-8") as f:
first = f.readline().rstrip("\n")
cols = [c.strip().lower() for c in first.split(",")]
if "name" in cols and ("file" in cols or "path" in cols):
name_idx = cols.index("name")
file_idx = cols.index("file") if "file" in cols else cols.index("path")
else:
name_idx, file_idx = 0, 1
parts = first.split(",")
if len(parts) > 1:
m[parts[0].strip()] = parts[1].strip()
for line in f:
parts = line.rstrip("\n").split(",")
if len(parts) <= max(name_idx, file_idx):
continue
m[parts[name_idx].strip()] = parts[file_idx].strip()
return m

# ---------- load edges ----------
print(f"Loading edges from {CSV_FILE} ...")
adj = defaultdict(list)
known_nodes = set()
kind_of = {}
csv_paths = {}

with open(CSV_FILE, "r", encoding="utf-8") as f:
header_line = f.readline().rstrip("\n")
header_cols = [c.strip() for c in header_line.split(",")]
header_lower = [c.lower() for c in header_cols]

src_idx  = _index_of(header_lower, SRC_KEYS, 0)
tgt_idx  = _index_of(header_lower, TGT_KEYS, 1)
kind_idx = _index_of(header_lower, KIND_KEYS, None)
file_idxs = _indices_of(header_lower, FILE_KEYS)

print(f"  Header: {header_cols}")
print(f"  -> source column: '{header_cols[src_idx]}' (index {src_idx})")
```

```
print(f"  -> target column: '{header_cols[tgt_idx]}' (index {tgt_idx})")
if kind_idx is not None:
print(f"  -> kind column:   '{header_cols[kind_idx]}' (index {kind_idx})")
else:
print("  -> kind column:   not present (heuristic will be used)")
if file_idxs:
names = ", ".join(f"'{header_cols[i]}'" for i in file_idxs)
print(f"  -> file columns:  {names}")
else:
print("  -> file columns:  none detected")

for line in f:
parts = line.rstrip("\n").split(",")
if len(parts) <= max(src_idx, tgt_idx):
continue
src = parts[src_idx].strip()
tgt = parts[tgt_idx].strip()
adj[src].append(tgt)
known_nodes.add(src)
known_nodes.add(tgt)

if kind_idx is not None and len(parts) > kind_idx:
raw = parts[kind_idx].strip().lower()
if raw in ("def", "definition", "abbrev", "instance", "structure"):
kind_of[src] = "def"
elif raw in ("thm", "theorem", "lemma", "prop", "proposition",
"corollary"):
kind_of[src] = "thm"
elif raw in ("ax", "axiom"):
kind_of[src] = "ax"

for fi in file_idxs:
if len(parts) <= fi:
continue
val = parts[fi].strip()
if not val:
continue
col_name = header_lower[fi]
if "target" in col_name:
csv_paths.setdefault(tgt, val)
else:
csv_paths.setdefault(src, val)

edges_total = sum(len(v) for v in adj.values())
print(f"  {len(adj):,} source nodes, {edges_total:,} edges")
print(f"  {len(kind_of):,} nodes carry kind info from CSV")
print(f"  {len(csv_paths):,} nodes carry exact file paths from CSV")

PATHS_MAP = load_paths_file(PATHS_FILE)
for k, v in PATHS_MAP.items():
csv_paths[k] = v
if PATHS_MAP:
print(f"  {len(PATHS_MAP):,} additional mappings loaded from {PATHS_FILE}")

if TARGET not in known_nodes:
print(f"WARNING: '{TARGET}' not found in graph - tree may be empty.")

# ---------- path resolution ----------
def path_of(name):
if name in csv_paths:
return csv_paths[name]
best = None
for k in csv_paths:
if name.startswith(k + ".") and (best is None or len(k) > len(best)):
best = k
if best is not None:
return csv_paths[best]
# axioms live in Lean core / Mathlib's Logic.lean
```

```
if name in LEAN_AXIOMS:
if name in ("Classical.choice", "classical.choice",
"Classical.propComplete"):
return "Mathlib/Logic/Choice.lean (Lean core)"
return "Mathlib/Logic/Basic.lean (Lean core)"
best = None
for k in NAMESPACE_PATHS:
if (name == k or name.startswith(k + ".")) and \
(best is None or len(k) > len(best)):
best = k
if best is not None:
return NAMESPACE_PATHS[best]
if "." in name:
top = name.split(".", 1)[0]
if top in NAMESPACE_PATHS:
return NAMESPACE_PATHS[top]
if "." not in name or name.split(".", 1)[0] in LEAN_CORE:
return "Lean core (not in Mathlib)"
return f"Mathlib/**/{name.split('.', 1)[0]}/**"

# ---------- classification ----------
_INHERITANCE_RE = re.compile(r"\.to[A-Z]")

def classify_heuristic(name):
# axiom check comes first
if name in LEAN_AXIOMS:
return "ax"
if not name:
return "def"
last = name.rsplit(".", 1)[-1]
if last.startswith("inst") or ".inst" in name:
return "def"
if _INHERITANCE_RE.search(name):
return "def"
if last and last[0].isupper():
return "def"
if "_" in last:
return "thm"
return "def"

def node_kind(name):
if name in kind_of:
return kind_of[name]
return classify_heuristic(name)

def tag(name):
"""Class tag only; the path is emitted on its own line."""
if not CLASSIFY:
return ""
return f"  [{node_kind(name)}]"

def keep(name):
"""Return True if this node should be kept under the active filters."""
k = node_kind(name)
if NO_AXIOMS and k == "ax":
return False
if not (ONLY_THMS or ONLY_DEFS):
return True
if ONLY_THMS:
# keep theorems AND axioms: axioms are the foundation of proofs
return k in ("thm", "ax")
if ONLY_DEFS:
return k == "def"
return True

# ---------- tree data structure ----------
class TNode:
__slots__ = ("name", "children", "marker", "is_cycle")
```

```
def __init__(self, name):
self.name = name
self.children = []
self.marker = ""
self.is_cycle = False

_nodes_created = 0  # respect --max-nodes

def build_tree(node, depth=0, visited=None, seen_on_path=None):
global _nodes_created
if visited is None:
visited = set()
if seen_on_path is None:
seen_on_path = set()

n = TNode(node)

if node in seen_on_path:
n.is_cycle = True
return n
if DEDUP and node in visited:
n.marker = "(*)"
return n

if MAX_NODES is not None and _nodes_created >= MAX_NODES:
n.marker = "(node budget)"
return n
_nodes_created += 1

if DEDUP:
visited.add(node)
seen_on_path.add(node)

if MAX_DEPTH is not None and depth >= MAX_DEPTH:
n.marker = "(depth limit)"
seen_on_path.discard(node)
return n

deps = adj.get(node, [])
if not deps:
n.marker = LEAF_MARKER if LEAF_MARKER else ""
seen_on_path.discard(node)
return n

for dep in sorted(set(deps)):
n.children.append(build_tree(dep, depth + 1, visited, seen_on_path))

seen_on_path.discard(node)
return n

def prune_tree(node):
if not keep(node.name):
return None
new = TNode(node.name)
new.marker = node.marker
new.is_cycle = node.is_cycle
for child in node.children:
pc = prune_tree(child)
if pc is not None:
new.children.append(pc)
return new

def render_struct(node, prefix="", is_root=True, is_last=True):
"""Two lines per node: declaration + tag, then ‘@ PATH‘.

The second line reuses the same ancestor ‘prefix‘ as the first
so the ‘@‘ sits flush under the branch marker.
"""
```

```
lines = []
display = f"(cycle: {node.name})" if node.is_cycle else node.name

# ---- name line ------------------------------------------------
if is_root:
name_line = display + tag(node.name)
else:
name_line = (prefix
+ ("`-- " if is_last else "|-- ")
+ display
+ tag(node.name))
if node.marker:
name_line += f"   {node.marker}"
lines.append(name_line)

# ---- path line ------------------------------------------------
if SHOW_PATHS:
lines.append(prefix + "@ " + path_of(node.name))

# ---- children -------------------------------------------------
child_prefix = "" if is_root else prefix + ("    " if is_last else "|   ")
n = len(node.children)
for i, child in enumerate(node.children):
lines.extend(render_struct(
child, prefix=child_prefix, is_root=False,
is_last=(i == n - 1)))
return lines

def count_nodes(node):
total = 1
for c in node.children:
total += count_nodes(c)
return total

# ---------- main ----------
print(f"Generating dependency tree for: {TARGET}")
if UNLIMITED:
print("  --depth 0 (unlimited expansion)")
if MAX_NODES is not None:
print(f"  --max-nodes {MAX_NODES:,}")
if not DEDUP:
print("  --no-dedup (subtrees re-expanded at every occurrence)")

root = build_tree(TARGET, depth=0)
raw_size = count_nodes(root)
budget_hit = MAX_NODES is not None and _nodes_created >= MAX_NODES
print(f"  raw tree nodes: {raw_size:,}")
if budget_hit:
print(f"  WARNING: node budget ({MAX_NODES:,}) reached; tree was truncated.")

if not DEDUP and raw_size > 200_000:
print(f"  WARNING: --no-dedup produced {raw_size:,} nodes; "
f"the output file may be very large.")

if ONLY_THMS or ONLY_DEFS or NO_AXIOMS:
pruned = prune_tree(root)
if pruned is None:
pruned = TNode(root.name)
pruned.marker = "(filtered out: target does not match --only-* filter)"
root = pruned

lines = render_struct(root)

n_def = sum(1 for ln in lines if "[def]" in ln)
n_thm = sum(1 for ln in lines if "[thm]" in ln)
n_ax  = sum(1 for ln in lines if "[ax]"  in ln)
n_leaf = sum(1 for ln in lines if LEAF_MARKER and LEAF_MARKER in ln)
```

```
header = [
"=" * 60,
f"Dependency tree for: {TARGET}",
f"Source: {CSV_FILE}",
f"Max depth: {'unlimited' if UNLIMITED else MAX_DEPTH}",
"(*) = already expanded elsewhere in this tree",
]
if CLASSIFY:
header.append("[def] = definition / typeclass instance")
header.append("[thm] = theorem / lemma")
header.append("[ax]  = Lean kernel axiom "
"(propext, funext, Quot.sound, Classical.choice)")
if SHOW_PATHS:
if csv_paths:
header.append("@ PATH = on the line below each declaration, "
"exact from mathlib_edges.csv (or --paths) "
"where available; heuristic otherwise")
else:
header.append("@ PATH = on the line below each declaration, "
"heuristic from namespace only "
"(no path columns in CSV, no --paths file)")
if LEAF_MARKER:
header.append(f"{LEAF_MARKER} = node has no further dependencies "
f"in the CSV")
if not DEDUP:
header.append("Dedup: disabled (subtrees re-expanded)")
if ONLY_THMS:
header.append("Filter: --only-thms "
"(pruned [def] subtrees; [ax] retained)")
elif ONLY_DEFS:
header.append("Filter: --only-defs (pruned [thm]/[ax] subtrees)")
if NO_AXIOMS:
header.append("Filter: --no-axioms (dropped [ax] nodes)")
if MAX_NODES is not None:
header.append(f"Node budget: {MAX_NODES:,}"
+ ("  [HIT - output truncated]"
if budget_hit else ""))
header.append("=" * 60)
header.append("")

with open(OUT_FILE, "w", encoding="utf-8") as f:
f.write("\n".join(header + lines))
f.write("\n")

print(f"Done. Written to: {OUT_FILE}")
print(f"  {len(lines):,} lines")
if CLASSIFY:
print(f"  {n_def:,} def, {n_thm:,} thm, {n_ax:,} ax")
if LEAF_MARKER:
print(f"  {n_leaf:,} terminal nodes ({LEAF_MARKER})")
```

7. Tables

7.1 Timeline

Key dates in the formalisation of FTC [3, 4].

2013	**first Lean 2 formalisation**	**integral_deriv_eq_sub (Lean 2 name)**	**Hoel et al.**
2017	**Lean 3 port**	**integral_eq_sub_of_hasDerivAt**	**mathlib3**

2021	interval integral rewritten	FTCFilter introduced	mathlib3 PR
2022	Mathlib 4 port	mathlib4 naming convention	mathlib4
2024	general Little-o FTC-2 stabilised	measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae	mathlib4

7.2 Classifications

Every node of the depth-5 tree falls into one of four classes.

Ax	2 visible at depth 5 in excerpt; 3 reached overall	funext, Quot.sound; propext (supplement)	kernel axioms
Def	dominant class	IntervalIntegrable, FTCFilter, NormedAddCommGroup	typeclass closure
Lem	many technical steps	IsLittleO.add_add, Filter.mp_mem	technical steps
Thm	$\approx$ 25 substantive nodes	measure_integral_sub_linear_isLittleO_of_tendsto_ae	mathematical content

7.3 Applications

The general Little-o form of FTC-2 above underpins:

App 1	change of variables	integral_comp_mul_deriv	Change of Variables
App 2	inverse integration / differentiation	integral_eq_sub_of_hasDerivAt	Barrow's Rule
App 3	FTC as named target	integral_deriv_eq_sub	Wiedijk Thm 14
App 4	computes π	integral_sqrt_one_sub_sq	Area of the Circle
App 5	E-valued Bochner generalisation	MeasureTheory.integral $[CompleteSpace\ E]$	Bochner Integral FTC

8. Conclusion

The Fundamental Theorem of Calculus in Mathlib is not a single theorem but a small hierarchy whose most general member is the Little-o statement intervalIntegral.measure_integral_sub_integral_sub_linear_isLittleO_of_tendsto_ae. From it one derives the classical derivative form intervalIntegral.integral_eq_sub_of_hasDerivAt and the one-sided variants _left and _right. The dependency tree in Section 6.1 makes this structure explicit.

The depth-5 closure of the one-sided theorem contains on the order of 10^5 nodes. Two of the three kernel axioms reachable in the closure — funext and Quot.sound — are visible in the printed excerpt; propext appears immediately beyond it on the Asymptotics.IsLittleO.sub branch. Classical .choice is not reached. The same methodology — extract the dependency tree, cross-check signatures against the documentation [5], and inspect the kernel-axiom closure — applies to any other theorem in the library [4, 8]. The Lean skeleton of Section 6.2 has been type-checked against Mathlib commit `0c1d5a7e` and introduces no new axiom dependencies beyond those already present in the one-sided statement.

References

[1] Larsen James Close. The Axiom Profile of Mathlib. Zenodo, 2026. DOI: 10.5281/zenodo.20710920. https://zenodo.org/records/20710921.

[2] DeepSeek. DeepSeek Chat. Conversational assistant used to assemble the exposition, cross-check the statement signatures against the Mathlib source, and prepare the LaTeX source of this paper. https://chat.deepseek.com.

[3] Lean community. Mathlib statistics. https://leanprover-community.github.io/mathlib_stats.html.

[4] The mathlib Community. The Lean mathematical library. Source repository, GitHub. https://github.com/leanprover-community/mathlib4.

[5] The mathlib Community. Mathlib4 documentation browser. Online reference for every declaration in Mathlib; used to locate the signatures and dependency edges of the theorems in this paper, and recommended as the entry point for analogous dependency-guided expositions of other theorems in the same library. https://leanprover-community.github.io/mathlib4_docs/.

[6] The mathlib Community. Mathlib/MeasureTheory/Integral/IntervalIntegral/FundThmCalculus.lean. Mathlib4 source file containing the two-sided Little-o FTC-2 and the one-sided reduction used in Section 5 and Section 6. Pinned to commit 0c1d5a7e. https://github.com/leanprover-community/mathlib4/blob/0c1d5a7e/Mathlib/MeasureTheory/Integral/IntervalIntegral/FundThmCalculus.lean.

[7] Xinze Li and Patrick Shafto. The Multinetwork of Mathlib. NeurIPS 2026 Datasets and Benchmarks Track, 2026. Under review. Dataset file: mathlib_edges.csv. https://huggingface.co/datasets/MathNetwork/MathlibGraph/resolve/main/mathlib_edges.csv.

[8] Xinze Li and Patrick Shafto. MathlibGraph: The Multinetwork of Mathlib. Hugging Face dataset, 2026. https://huggingface.co/datasets/MathNetwork/MathlibGraph.

[9] Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The Network Structure of Mathlib. arXiv:2604.24797, 2026. https://arxiv.org/abs/2604.24797.